Scraping + Chunking – Design

This document describes the two core processes that together form the FSR (Field Service Report) data pipeline:

1. **Process 1 – Scraping & Metadata Extraction**
2. **Process 2 – Chunking & Vector Embedding**

Both processes run on Databricks and operate on the same set of PDF source volumes.

Process 1 – Scraping & Metadata Extraction

Purpose

Scan FSR PDF files from Unity Catalog volumes, extract structured metadata fields from each PDF, normalise them using an LLM, enrich with IBAT / Event Vision lookup tables, and write the results to a table.

Input

Source Volumes

Files are read non-recursively from the following Unity Catalog volume paths:

#	Volume Path	Catalog
1	`/Volumes/viud/ing_ud_fsr_manual/manual_field_service_report/ecrt_reports`	`viud`
2	`/Volumes/viud/ing_ud_fieldvision/fv_field_service_report`	`viud`

File Types

- **Accepted:** `.pdf` files at the top level of each volume path
- **Skipped:** `.crdownload`, `.tmp`, `.part`, `.download`, `.DS_Store`, `.json`, `.txt`, `.csv`, `.xlsx`, and any sub-directories

Lookup / Enrichment Sources

Source	Default Location	Purpose
IBAT Equipment	`vgpd.prm_std_views.ibat_equipment_mst`	ESN / equipment type enrichment
Event Vision SOT	`vgpd.fsr_std_views.eventmgmt_event_vision_sot`	Event / project ID enrichment

New File Detection (File Registry + Status Tracking)

Process 1 uses a dedicated **file registry table** to track every file discovered on the volumes and its scraping status. Failed scrapes are automatically retried on the next run.

File Registry Table

`vaid/vaiq/vaip.ai_sot_field_service_report.fsr_file_registry`

Column	Type	Description
`volume_path`	STRING	**Primary key.** Full volume path to the PDF file
`pdf_name`	STRING	File name only
`last_modified`	TIMESTAMP	File modification timestamp from `dbutils.fs.ls`
`scraped`	STRING	`pending` = not yet scraped, `completed` = scraped successfully, `failed` = scraping failed
`error_reason`	STRING	Error message if scraping failed, otherwise null
`discovered_at`	TIMESTAMP	When the file was first registered
`scraped_at`	TIMESTAMP	When scraping last completed or failed

Step 1 – Discovery (file registry sync)

1. Query `MAX(discovered_at)` from the registry table as a watermark
2. Scan volumes via `dbutils.fs.ls`, filtering to files where `last_modified > watermark`
3. **Upsert** each new/changed file into the registry:
    - **New file** (`volume_path` not in registry) → insert with `scraped = 'pending'`
    - **Re-uploaded file** (`volume_path` exists but `last_modified` changed) → update row, reset `scraped = 'pending'` — the changed timestamp triggers a re-scrape automatically
    - **Unchanged file** → no action

Step 2 – Scraping (status-driven)

Query the registry for files to process:

```
SELECT * FROM fsr_file_registry
WHERE scraped = 'pending' OR scraped = 'failed'
```

This covers both new files and any previously failed ones in a single query — no separate retry logic needed.

- **On success** → set `scraped = 'completed'`, `scraped_at = current_timestamp()`
- **On failure** → set `scraped = 'failed'`, `error_reason = <message>`, `scraped_at = current_timestamp()`

> ***Retry behaviour:*** *Files with `scraped = 'failed'` or `scraped = 'pending'` are retried on every run.*

> ***`FORCE_RESET` override:*** *Setting `FORCE_RESET = true` truncates both the registry table and the metadata output table, and re-registers all files from scratch.*

Processing Steps (high-level)

Step 1 – Registry sync (file discovery)

1. Query `MAX(discovered_at)` from the file registry table as a watermark (`null` = first run, scan all files)
2. Scan volumes via `dbutils.fs.ls`, filtering to files where `last_modified > watermark`
3. Upsert new/changed files into the registry with `scraped = 'pending'`

Step 2 – Scraping

1. Query registry: `WHERE scraped = 'pending' OR scraped = 'failed'`
2. Extract key-value fields and title from page 1 of each PDF
3. Send extracted text to LLM for field normalisation (model: `gemini-3-flash` by default)
4. Enrich normalised records with IBAT and Event Vision lookups
5. Append enriched rows to the metadata output table with `processed = pending` and `ingestion_timestamp = current_timestamp()`
6. Update registry row: `scraped = 'completed'` on success, `scraped = 'failed'` + `error_reason` on failure

Output

Output Table

`vaid/vaiq/vaip.ai_sot_field_service_report.biz_metadata_field_service_report`

Schema

Column	Type	Description
`title`	STRING	Report title extracted from the PDF
`customer`	STRING	Customer / site name
`esn`	STRING	Equipment Serial Number
`equipment_sys_id`	STRING	System ID from IBAT
`equipment_type`	STRING	Equipment type (e.g. Generator, Gas Turbine)
`equipment_code`	STRING	Equipment sub-class / code (lookup – reliability TBC)
`event_type`	STRING	Event type (e.g. Call-Out, Inspection)
`ev_project_id`	STRING	Event Vision project ID (lookup)
`ev_equipment_event_id`	STRING	Event Vision equipment event ID (lookup)
`ofs_event_id`	STRING	OFS event ID (lookup – validate with SMEs)

`fsp_project_id`	STRING	FSP project ID
`project_id`	STRING	Additional project ID field (pipe-separated if multiple)
`fsr_number`	STRING	FSR document number (lookup)
`report_issued_date`	STRING	Date the FSR was issued — **format: YYYY-MM-DD**
`outage_start_date`	STRING	Outage start date — **format: YYYY-MM-DD**
`outage_end_date`	STRING	Outage end date — **format: YYYY-MM-DD**
`prepared_by`	STRING	Name of the person who prepared the report
`approved_by`	STRING	Name of the person who approved the report
`pdf_name`	STRING	Source PDF file name / identifier
`volume_path`	STRING	Full volume path to the source PDF file (used by Process 2 to locate the file)
`processed`	STRING	Chunking status: `pending` = not yet processed, `completed` = chunked successfully, `failed` = chunking failed
`error`	STRING	Short error type / code if chunking failed, otherwise null
`error_reason`	STRING	Full error message / stack trace if chunking failed, otherwise null
`ingestion_timestamp`	TIMESTAMP	When this row was written by Process 1 — used as the watermark for incremental detection on the next run

> ***Date format:*** *All date columns must be normalised to `YYYY-MM-DD` (ISO 8601) during the LLM normalisation step, regardless of how they appear in the source PDF.*

Process 2 – Chunking & Vector Embedding

Purpose

Read FSR PDFs from the same volumes, parse and recursively chunk the full document content, generate vector embeddings, and write chunk rows to a Delta table backed by Databricks Vector Search.

Input

Source Table (Process 1 Output)

Process 2 does **not** scan volumes directly. It reads from the Process 1 output table and processes only rows where `processed` is `pending`, or `failed`.

Table	Purpose
`vaid/vaiq/vaip.ai_sot_field_service_report.biz_metadata_field_service_report`	Provides both the `volume_path` to locate each PDF and all metadata fields for enrichment

The `volume_path` column on each row is used to load the physical PDF file from the volume. No separate volume scan or enrichment table is required — all metadata is already available in the source table row.

> ***Incremental by design:*** *Because Process 2 filters on `processed` status, re-running the job automatically picks up any new or previously failed files without any additional configuration.*

Processing Steps (high-level)

1. Read all rows from the Process 1 metadata table where `processed` is `pending`, or `failed`
2. For each row, use `volume_path` to load the PDF file from the Unity Catalog volume
3. Extract full text from the PDF using `PyMuPDF`
4. Recursively split text into chunks (`langchain-text-splitters`, recursive strategy)
5. Denormalise metadata from the source row onto every chunk (no additional lookup needed)
6. Generate vector embeddings via LiteLLM (`azure-text-embedding-3-large-1`)
7. Write chunk rows to the Delta chunk table
8. Sync Delta table to Databricks Vector Search index
9. **On success:** update the source row — set `processed = 'completed'`
10. **On failure:** update the source row — set `processed = 'failed'`, populate `error` and `error_reason`; the row will be retried on the next run

Output

Output Delta Table

`vaid/vaiq/vaip.ai_std_con_field_service_report.vec_field_service_report`

Output Vector Search Index

`vaid/vaiq/vaip.ai_std_con_field_service_report.vs_vec_field_service_report`

Chunk Row Schema

Column	Type	Description
`chunk_id`	STRING	**Primary key.** Unique identifier per chunk — generated as `md5(pdf_name + chunk_index)`
`chunk_text`	STRING	Plain text content of the chunk
`embedding`	ARRAY<FLOAT>	Dense vector — generated by the configured embedding model (`azure-text-embedding-3-large-1`)
`metadata`	STRING (JSON)	All contextual fields serialised as a JSON object — see metadata fields below

`metadata` JSON Fields

All fields below are written into the `metadata` column as a single JSON object at chunking time, sourced directly from the Process 1 output table row:

Field	Description
`pdf_name`	Source PDF file name / identifier
`page_number`	Page number within the PDF where this chunk originates
`title`	Report title
`customer`	Customer / site name
`esn`	Equipment Serial Number
`equipment_sys_id`	System ID from IBAT
`equipment_type`	Equipment type (e.g. Generator, Gas Turbine)
`equipment_code`	Equipment sub-class / code
`event_type`	Event type (e.g. Call-Out, Inspection)
`ev_project_id`	Event Vision project ID
`ev_equipment_event_id`	Event Vision equipment event ID
`ofs_event_id`	OFS event ID
`fsp_project_id`	FSP project ID
`xxx_project_id`	Additional project ID (pipe-separated if multiple)
`fsr_number`	FSR document number
`report_issued_date`	Date the FSR was issued (`YYYY-MM-DD`)
`outage_start_date`	Outage start date (`YYYY-MM-DD`)
`outage_end_date`	Outage end date (`YYYY-MM-DD`)
`prepared_by`	Name of the person who prepared the report
`approved_by`	Name of the person who approved the report

Open Items / To Be Confirmed

- [ ] **LLM-generated document summary in `metadata`**